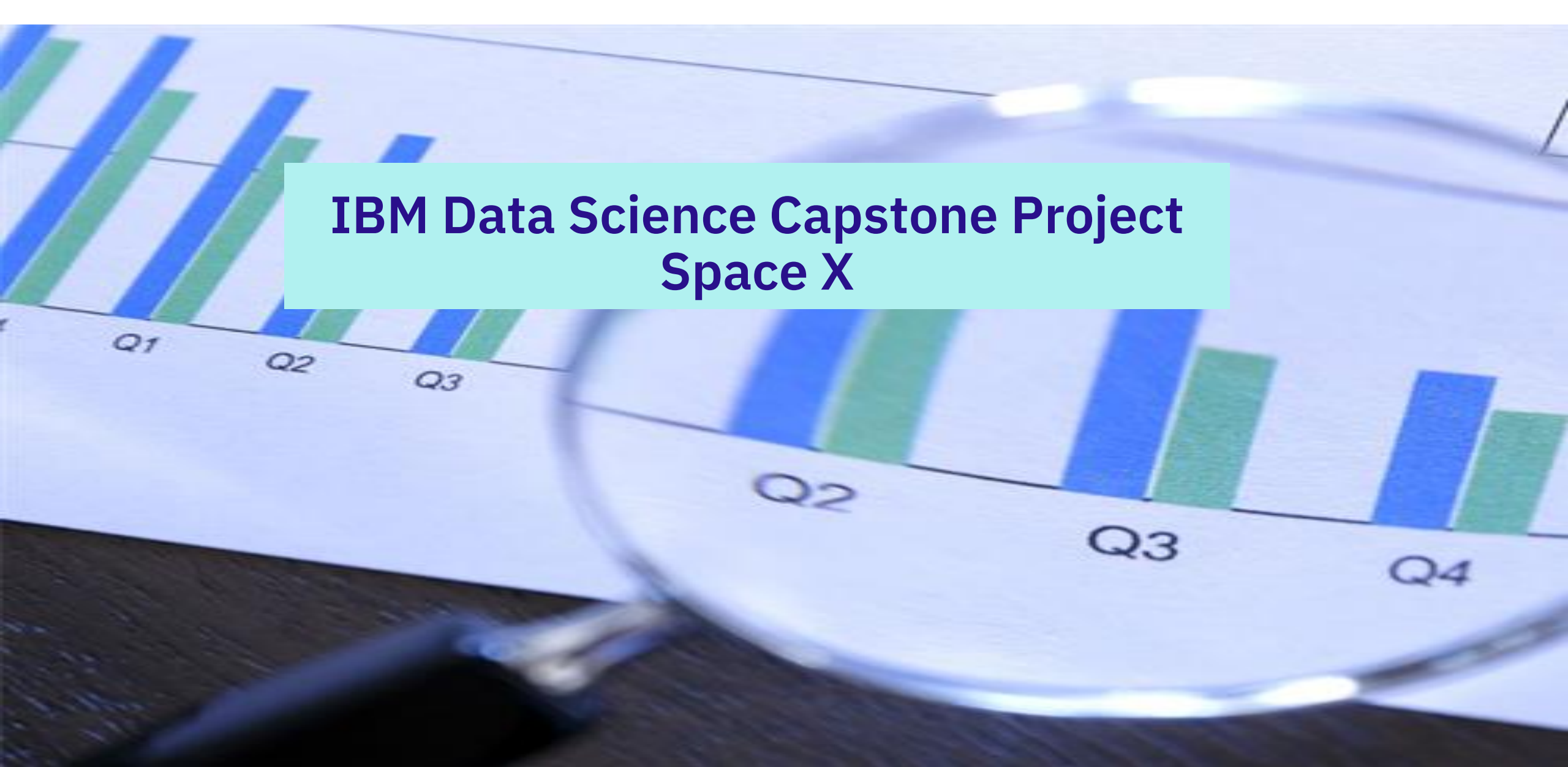




IBM Data Science Capstone Project Space X

Andrej Djukic

Space Exploration
Technologies



OUTLINE



- Executive Summary
- Introduction
- Methodology
- Results
 - Visualization – Charts
 - Dashboard
- Discussion
 - Findings & Implications
- Conclusion
- Appendix



EXECUTIVE SUMMARY



- Data Collection
- Data Wrangling
 - EDA with data visualization
 - EDA with SQL
 - Building an interactive map with Folium
- Building a Dashboard with Plotly Dash
- Predictive analysis (Classification)
- EDA results



INTRODUCTION



- **Project background and context**

We predicted if the Falcon 9 first stage will land successfully. SpaceX advertises Falcon 9 rocket launches on its website, with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage. Therefore, if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch.

- **Common problems that needed solving.**

What influences if the rocket will land successfully?

The effect each relationship with certain rocket variables will impact in determining the success rate of a successful landing.

What conditions does SpaceX have to achieve to get the best results and ensure the best rocket success landing rate.



METHODOLOGY

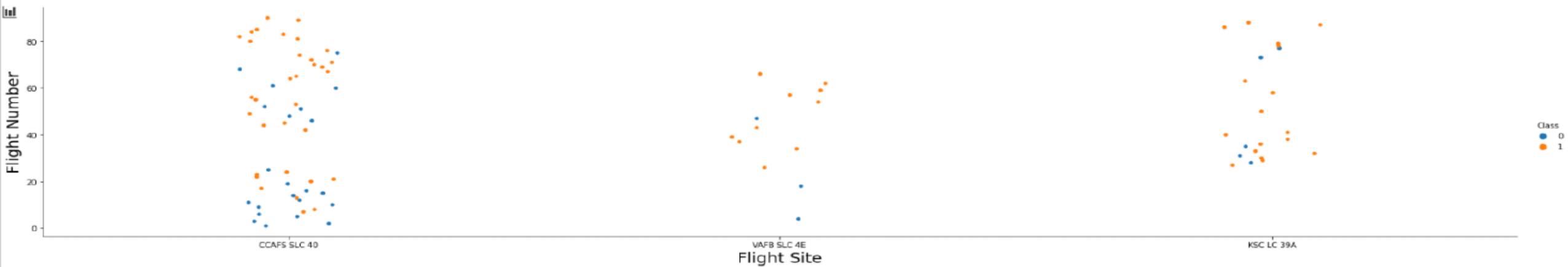


- Data collection methodology:
 - SpaceX Rest API
 - (Web Scrapping) from Wikipedia
- Performed data wrangling (Transforming data for Machine Learning)
One Hot Encoding data fields for Machine Learning and dropping irrelevant columns
- Performed exploratory data analysis (EDA) using visualization and SQL
Plotting : Scatter Graphs, Bar Graphs to show relationships between variables to show patterns of data.
- Performed interactive visual analytics using Folium and Plotly Dash
- Performed predictive analysis using classification models
 - How to build, tune, evaluate classification models



RESULTS

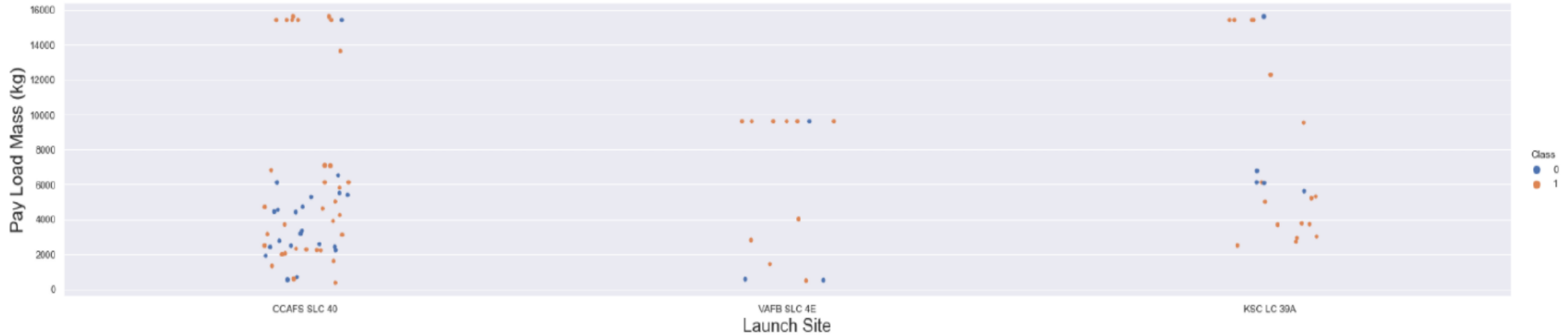
Flight Number vs. Flight Site



The more amount of flights at a launch site the greater the success rate at a launch site.



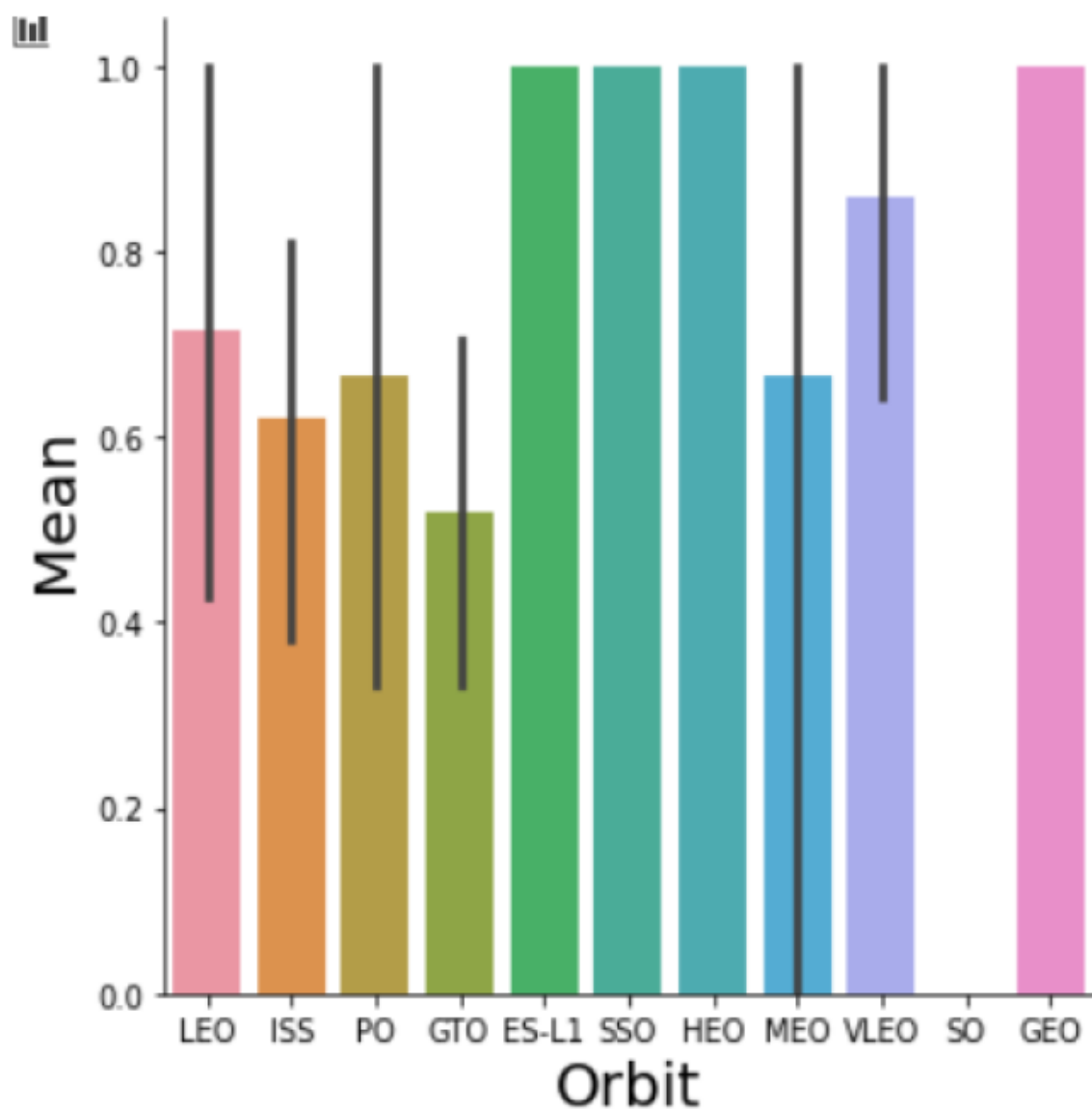
Payload Mass Vs Launch Site



The greater the payload mass for Launch Site CCAFS SLC 40 the higher the success rate for the Rocket. There is not quite a clear pattern to be found using this visualization to make a decision if the Launch Site is dependant on Pay Load Mass for a success launch.

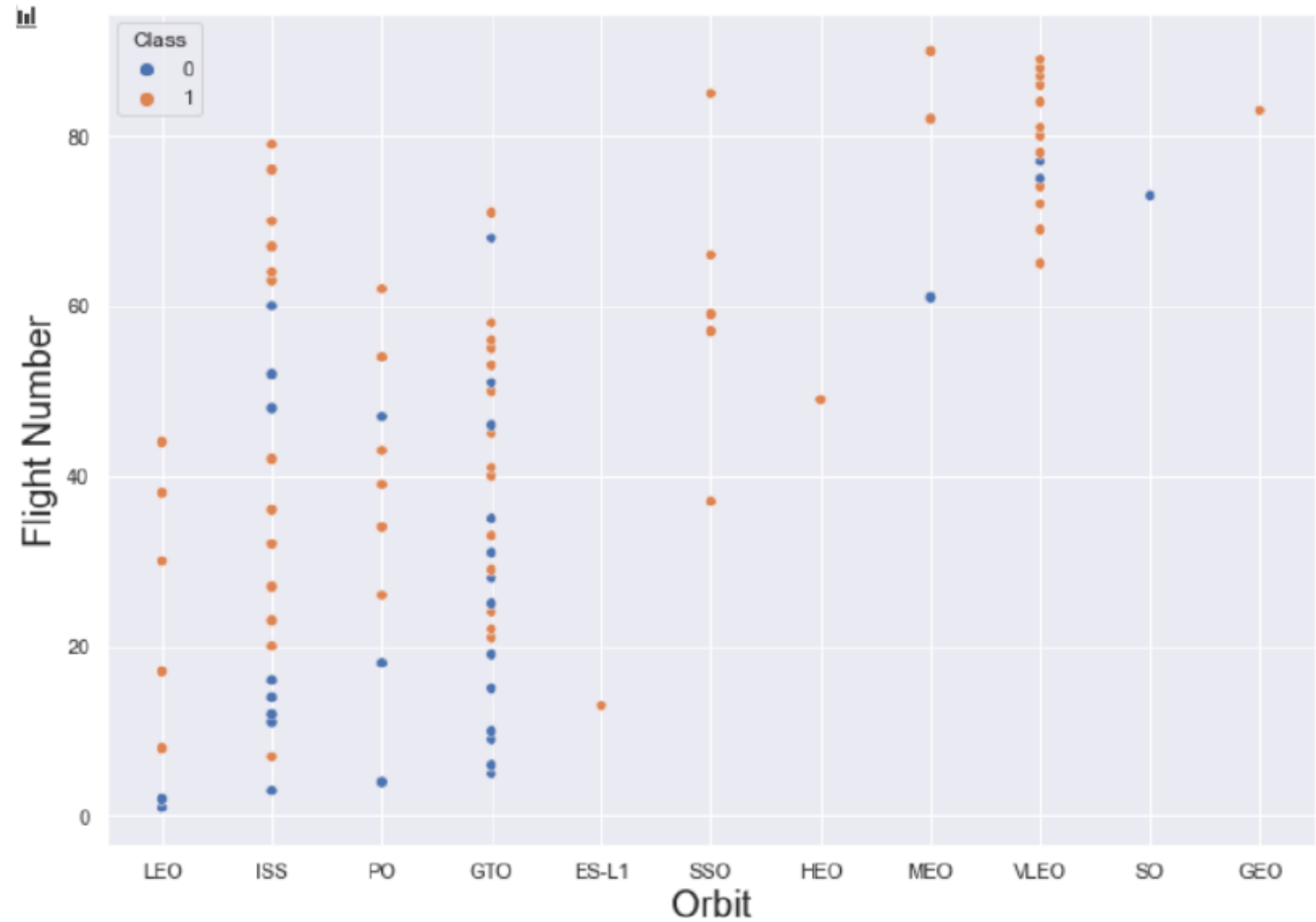
Success rate vs. Orbit type

Orbit GEO,HEO,SSO,ES-L1 has the best Success Rate



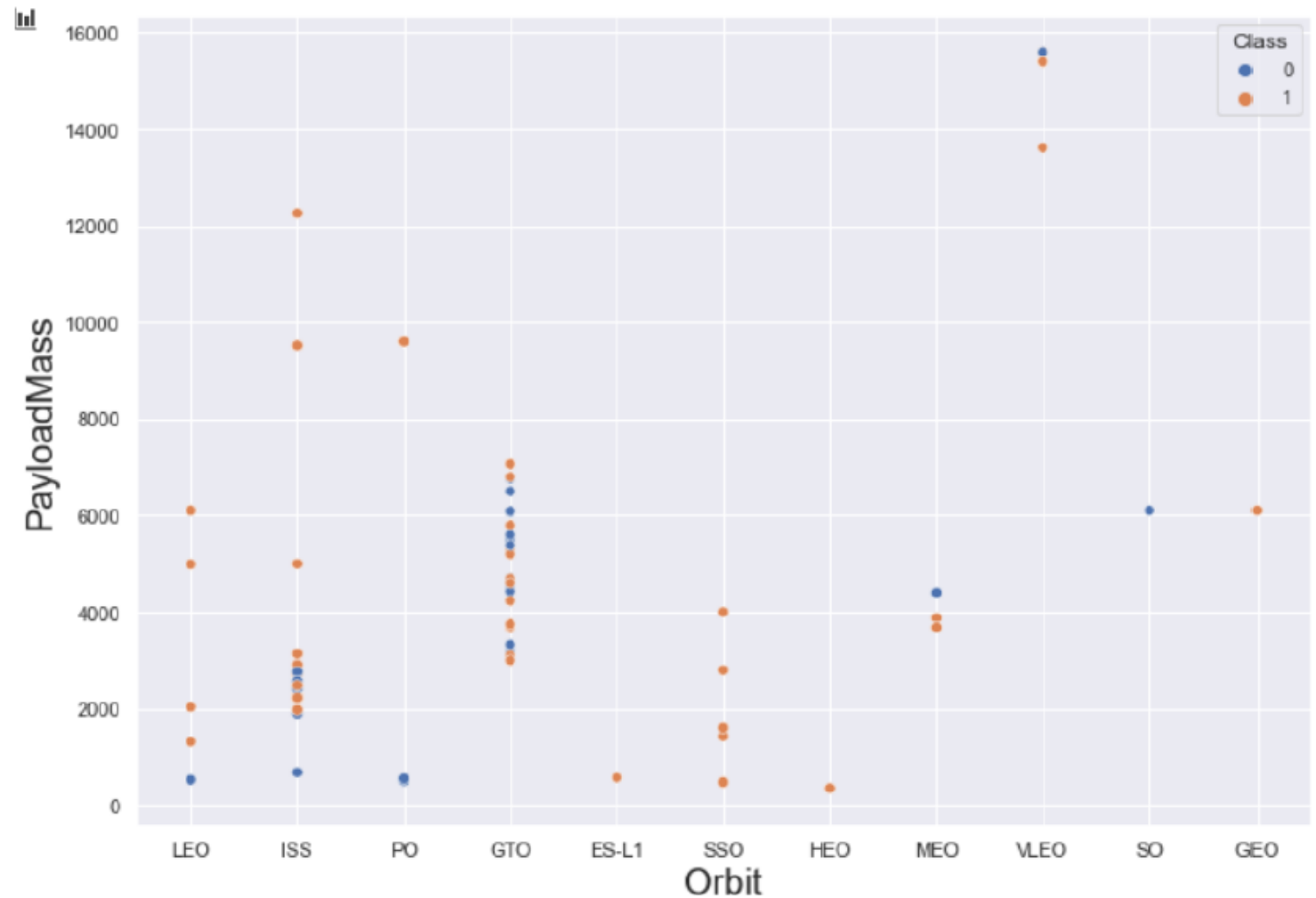
Flight Number vs. Orbit type

You should see that in the LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.



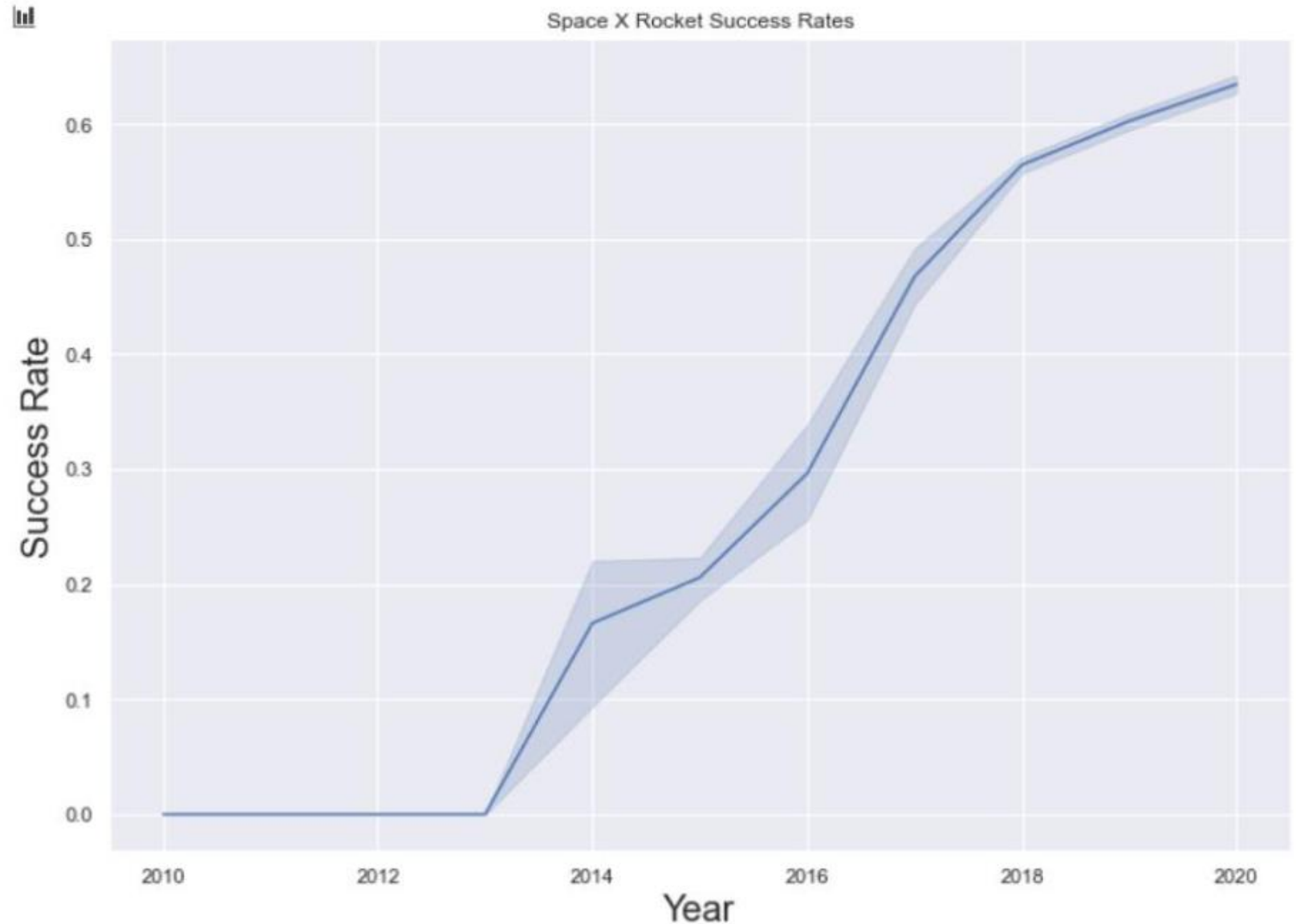
Payload vs. Orbit type

You should observe that Heavy payloads have a negative influence on GTO orbits and positive on GTO and Polar LEO (ISS) orbits.



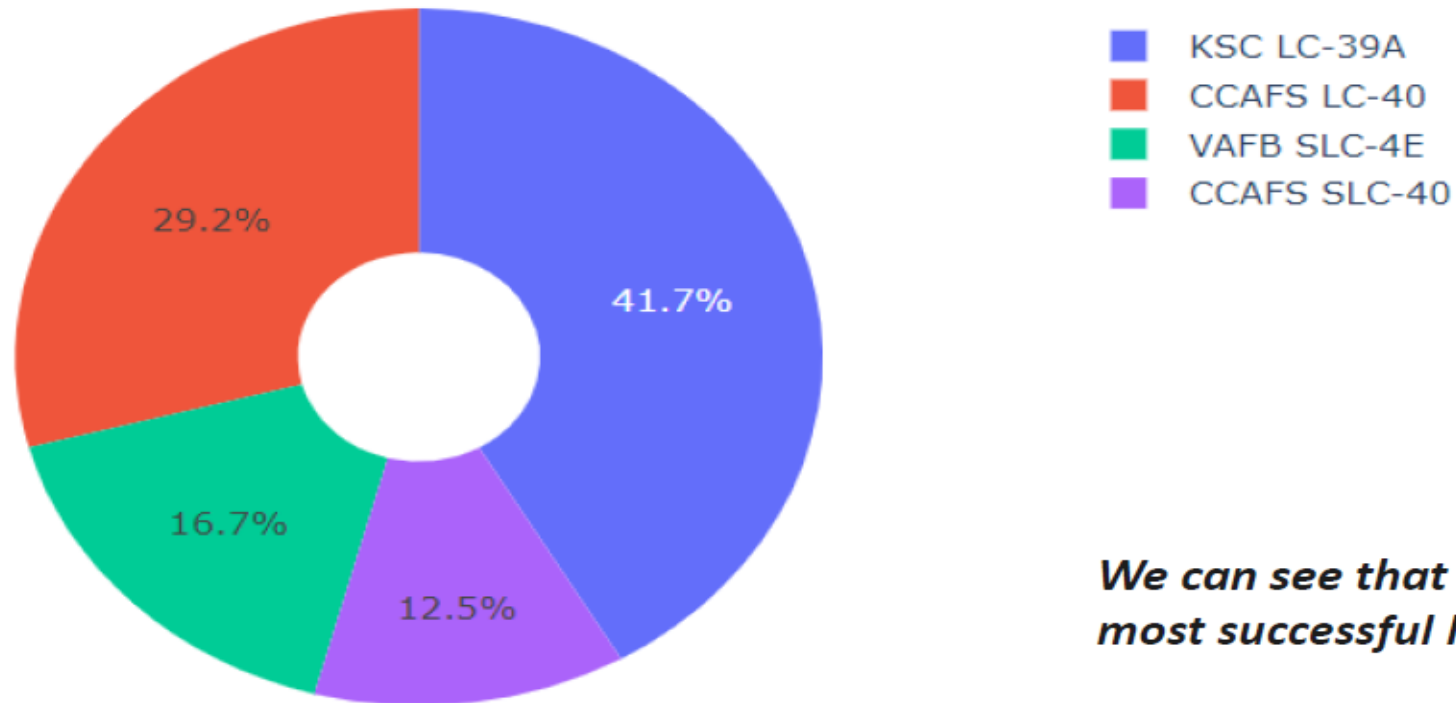
Launch success yearly trend

you can observe that the success rate since 2013 kept increasing till 2020



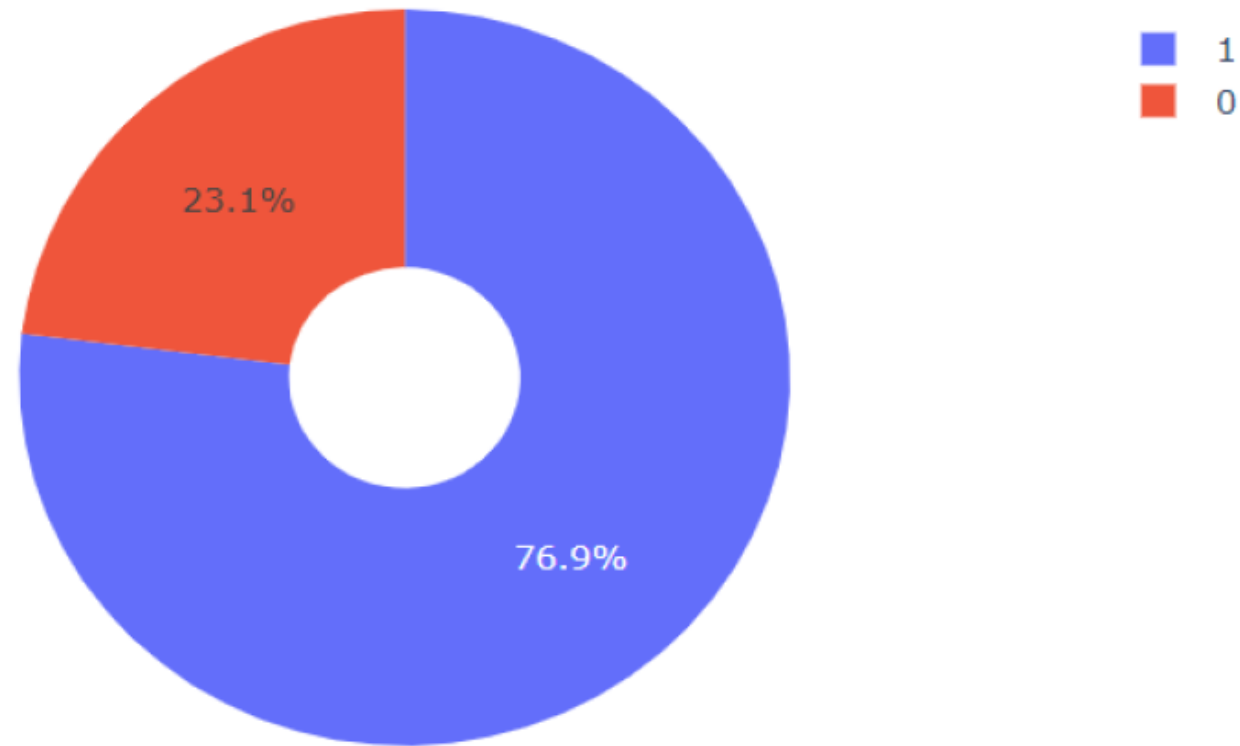
DASHBOARD TAB 1

Total Success Launches By all sites



We can see that KSC LC-39A had the most successful launches from all the sites

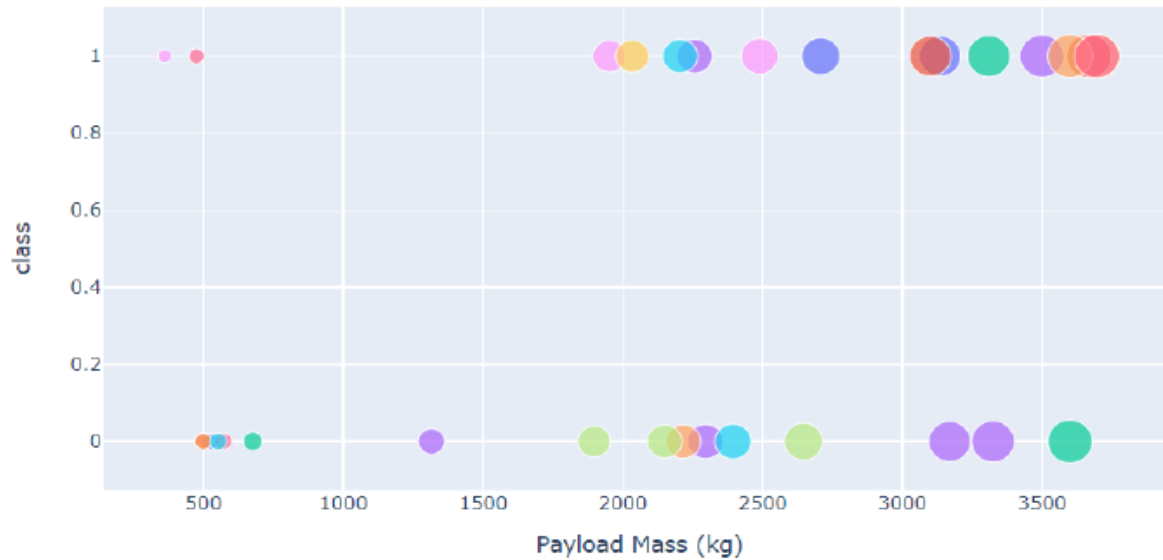
DASHBOARD TAB 2



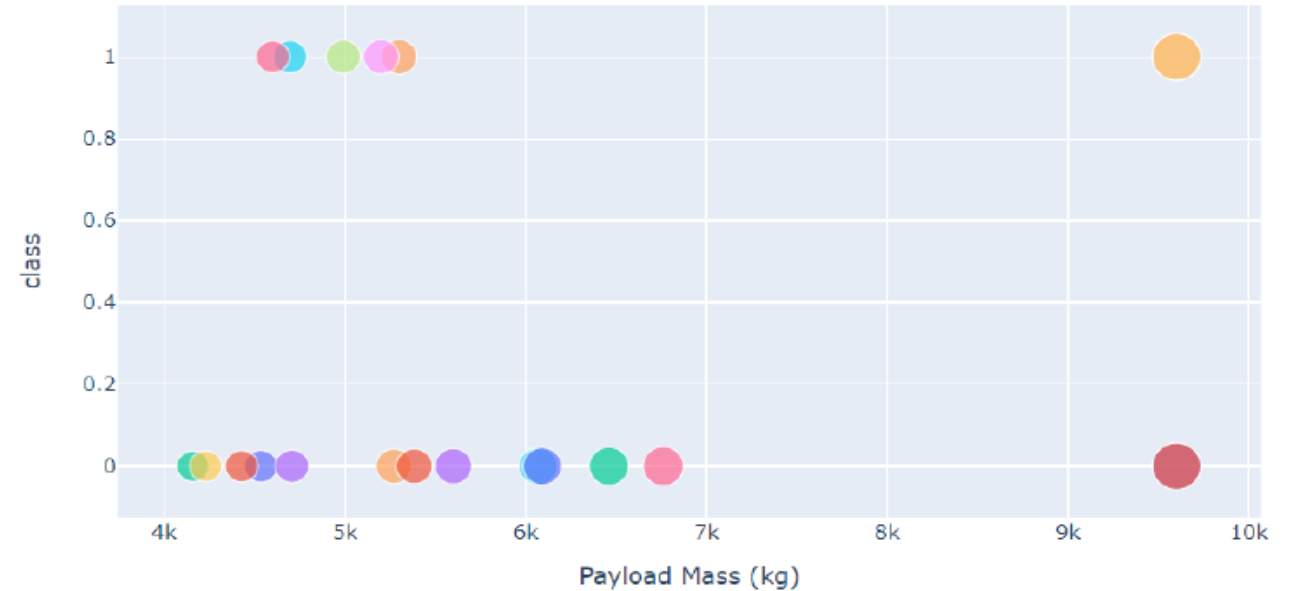
KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate

DASHBOARD TAB 3

Low Weighted Payload 0kg – 4000kg



Heavy Weighted Payload 4000kg – 10000kg



We can see the success rates for low weighted payloads is higher than the heavy weighted payloads

CONCLUSION



- The Tree Classifier Algorithm is the best for Machine Learning for this dataset
- Low weighted payloads perform better than the heavier payloads
- The success rates for SpaceX launches is directly proportional time in years they will eventually perfect the launches
- We can see that KSC LC-39A had the most successful launches from all the sites
- Orbit GEO,HEO,SSO,ES-L1 has the best Success Rate

APPENDIX



- Haversine formula
- ADGGoogleMaps Module (not used but created)

ADGGoogleMaps Module

Introduction

This is a python class I designed to make my life easier and others when getting coordinates from addresses and producing a Folium Python Map in a Jupyter Notebook. This class processing is powered by the Google Geocoding API.

Prerequisites

- Sign up for a Google Geocoding API
Set up a Google API Key
- Head to library under APIs & Services and add Geocoding API

Requirements

- Google API Secret (API Key)

Python Usage Documentation

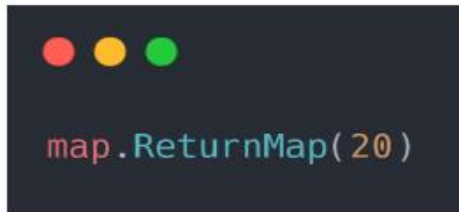
- Getting Cords

Simple arguments to follow by the object

`__init__(self, api_key, address)` – Look at following usage for guidance

```
import ADGGoogleMaps
map = ADGGoogleMaps.ADGGoogleMaps("google_api_secret_key", "your_address")
cords = map.GetCordsFromAddress()
cords
```

This returns a list with your coordinates of that address example
['longitude', 'latitude']



- Returning Folium Python Map in Jupyter Notebook

assuming you declared mapclass in 1.
`.ReturnMap(size)`

this function returns this Folium map in the Jupyter Notebook



Haversine Formula

Introduction

The haversine formula determines the great-circle distance between two points on a sphere given their longitudes and latitudes. Important in navigation, it is a special case of a more general formula in spherical trigonometry, the law of haversines, that relates the sides and angles of spherical triangles.

Usage

Why did I use this formula? First of all, I believe the Earth is round/elliptical. I am not a Flat Earth Believer! Jokes aside when doing Google research for integrating my [ADGGoogleMaps API](#) with a Python function to calculate the distance using two distinct sets of {longitudinal, latitudinal} list sets. Haversine was the trigonometric solution to solve my requirements above.

Formula

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

```
#Variables
```

```
d is the distance between the two points along a great circle of the sphere (see spherical distance),  
r is the radius of the sphere.
```

```
φ1, φ2 are the latitude of point 1 and latitude of point 2 (in radians),  
λ1, λ2 are the longitude of point 1 and longitude of point 2 (in radians).
```

